

6주차 (188~226p)

chapter 6 : 학습 관련 기술

6.1. 매개변수 갱신

신경망 학습의 목적 :

손실 함수의 값을 가능한 낮추는 매개변수를 찾는 것

즉, 최적값을 찾는 것

How? **최적화**를 이용하여

신경망의 최적화 :

최적의 매개변수를 찾기 위해 **기울기 (=미분)**을 사용

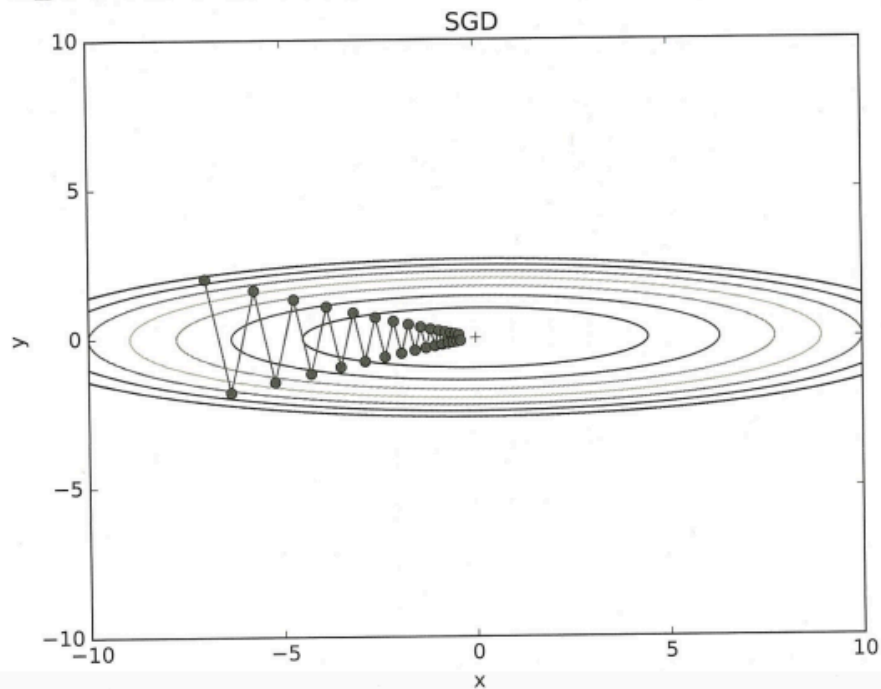
기울어진 방향으로 매개변수 값을 갱신

= SGD (확률적 경사 하강법)

SGD 단점 : '언젠간' 도달한다는 막연함

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}$$

그림 6-3 SGD에 의한 최적화 갱신 경로 : 최솟값인 (0, 0)까지 지그재그로 이동하니 비효율적이다.



제안된 코드에 라이선스가 적용될 수 있습니다. | KingXHJ/X-Readings

```
class SGD:
    def __init__(self, lr = 0.01):
        self.lr = lr
        #lr = learning rate (=학습률)
    def update(self, params, grads):
        for key in params.keys():
            params[key] -= self.lr * grads[key]
```

n은 학습률을 의미하며 값을 우리가 미리 정해서 어느정도의 값을 이동시킬지 지정

비등방성 함수라는 것이 단점이다

무작정 기울어진 방향에만 기대기보다 더 좋은 건 없을까?

SGD단점 개선

: 1) 모멘텀 2) AdaGrad 3) Adam

이렇게 총 3가지의 기법이 있다.

1. 모멘텀

: '운동량'

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \eta \frac{\partial L}{\partial \mathbf{W}}$$

$$\mathbf{W} \leftarrow \mathbf{W} + \mathbf{v}$$

변수 설명 :

W : 갱신할 가중치 매개변수

L을 W에 대하여 미분 : W에 대한 손실함수 기울기

n : 학습률

v : 속도

So, 기울기 방향으로 힘을 받아 물체가 가속되는 물리법칙을 나타냄

⇒ 아무런 힘을 받지 않으면 하강하게 되는 원리

```
import numpy as np

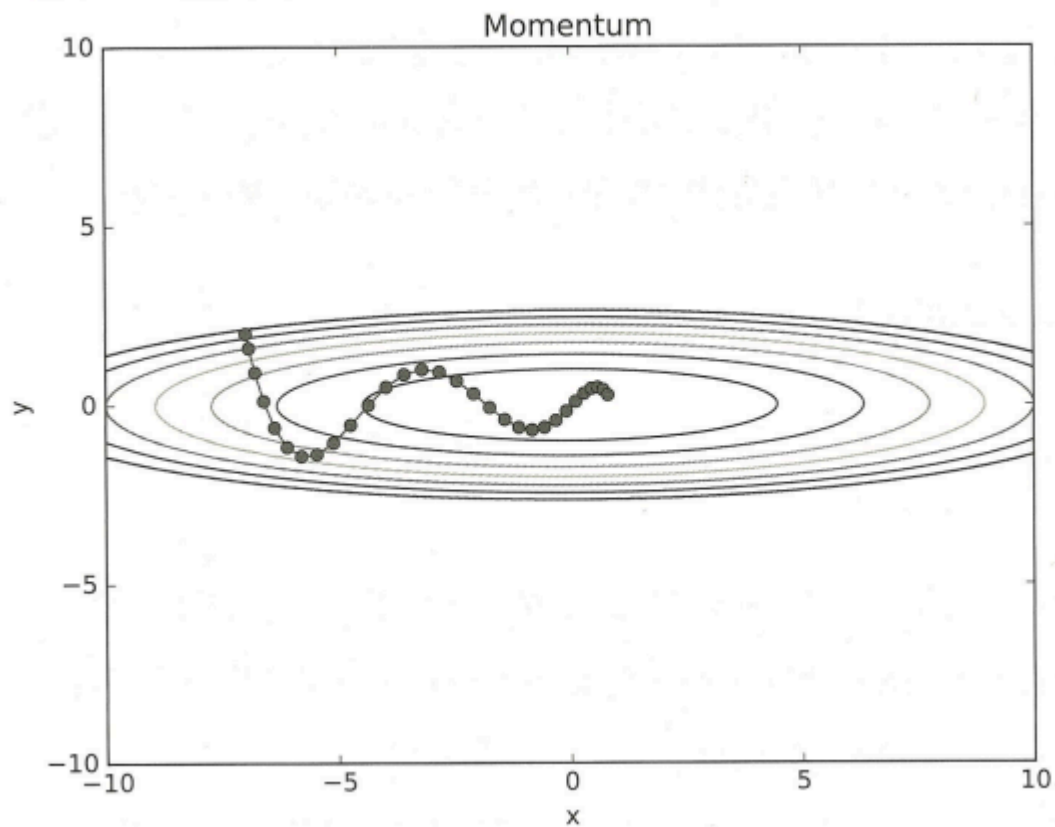
class Momentum:
    def __init__(self, lr=0.01, momentum=0.9):
        self.lr = lr
        self.momentum = momentum
        self.v = None

    def update(self, params, grads):
        if self.v is None:
            self.v = {}

        for key, val in params.items():
            self.v[key] = np.zeros_like(val)

        for key in params.keys():
            self.v[key] = self.momentum * self.v[key] - self.lr * grads[key]
            params[key] += self.v[key]
```

그림 6-5 모멘텀에 의한 최적화 갱신 경로



공이 바닥을 구르듯 움직이며 지그재그 정도가 덜하다.

2. AdaGrad

a. 신경망에서는 학습률이 중요

i. 학습률을 어떻게 정할까?

⇒ " 학습률 감소"

: 학습을 진행하면서 학습률을 점차 줄임 (사용률 높음)

$$\mathbf{h} \leftarrow \mathbf{h} + \frac{\partial L}{\partial \mathbf{W}} \odot \frac{\partial L}{\partial \mathbf{W}}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{\sqrt{\mathbf{h}}} \frac{\partial L}{\partial \mathbf{W}}$$

변수 설명 :

W : 갱신할 가중치 매개변수

L을 W에 대하여 미분 : W에 대한 손실함수 기울기

n : 학습률

h : **기존(과거) 기울기값을 제공하여 계속 더해주고, 1/루트 h** 를 곱해서 학습률 조정

→ 크게 갱신된 원소는 학습률이 낮아짐

→ 학습률 감소가 매개변수의 원소마다 다른 적용

```
class AdaGrad:

    """AdaGrad"""

    def __init__(self, lr=0.01):
        self.lr = lr
        self.h = None

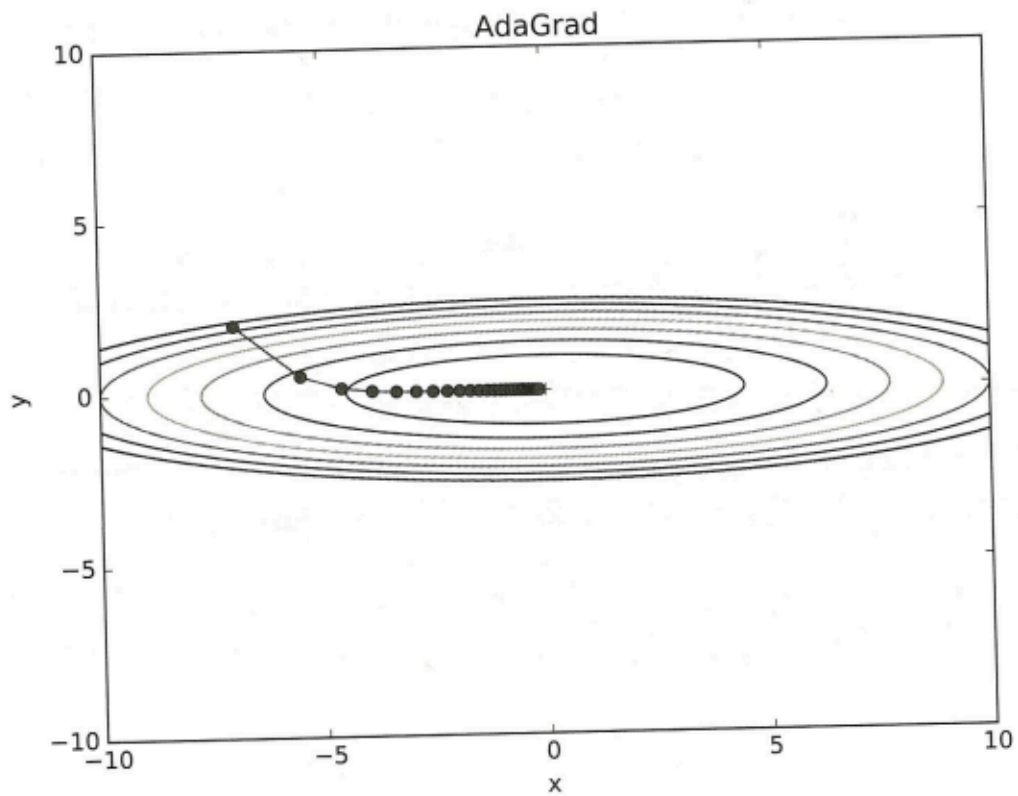
    def update(self, params, grads):
        if self.h is None:
            self.h = {}
            for key, val in params.items():
                self.h[key] = np.zeros_like(val)

        for key in params.keys():
            self.h[key] += grads[key] * grads[key]
            params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + 1e-7)
```

마지막줄에 1e-7이 포인트임

self.h[key]에 0이 담겨있어도 0으로 나누지 않도록 위험 사태를 방지한다.

그림 6-6 AdaGrad에 의한 최적화 갱신 경로



처음엔 큰 움직임이지만 움직임에 비례하여 갱신 정도도 큰 폭으로 작아지게 조정된다.
⇒ 지그재그 움직임 감소

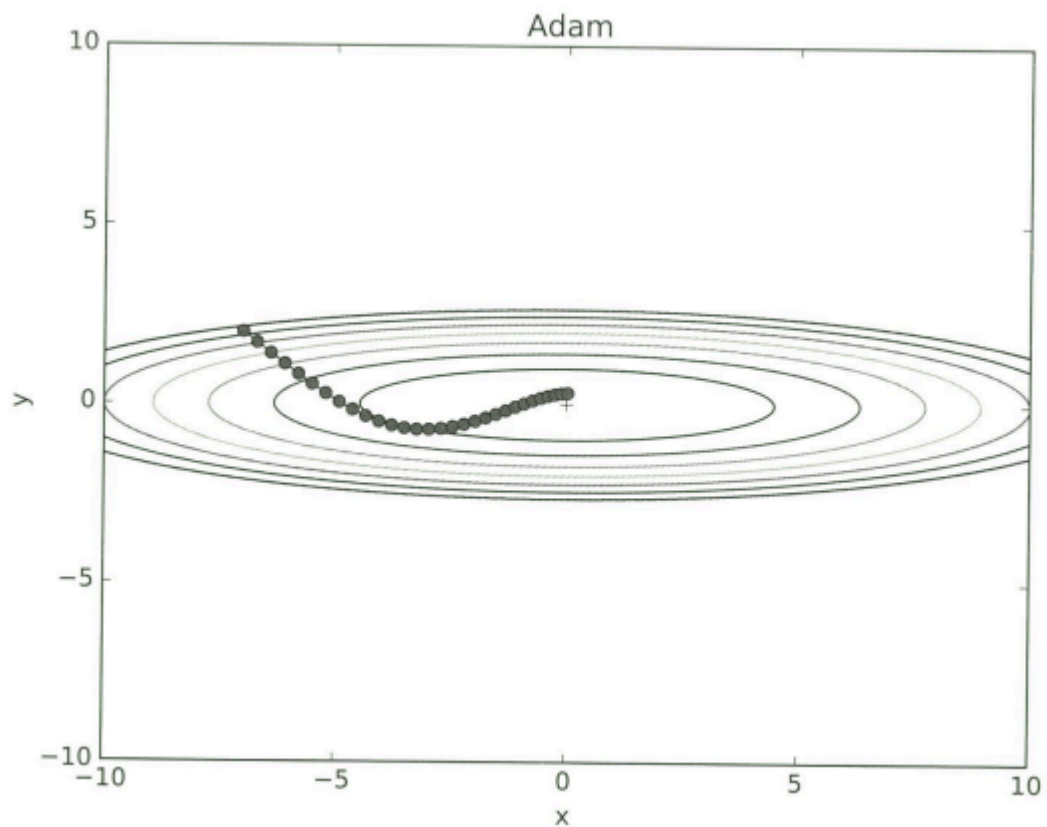
3. Adam

= 모멘텀(속도) + AdaGrad(학습률 자동조정)

⇒ 매개변수의 공간을 효율적으로 탐색

⇒ 편향 보정이 진행

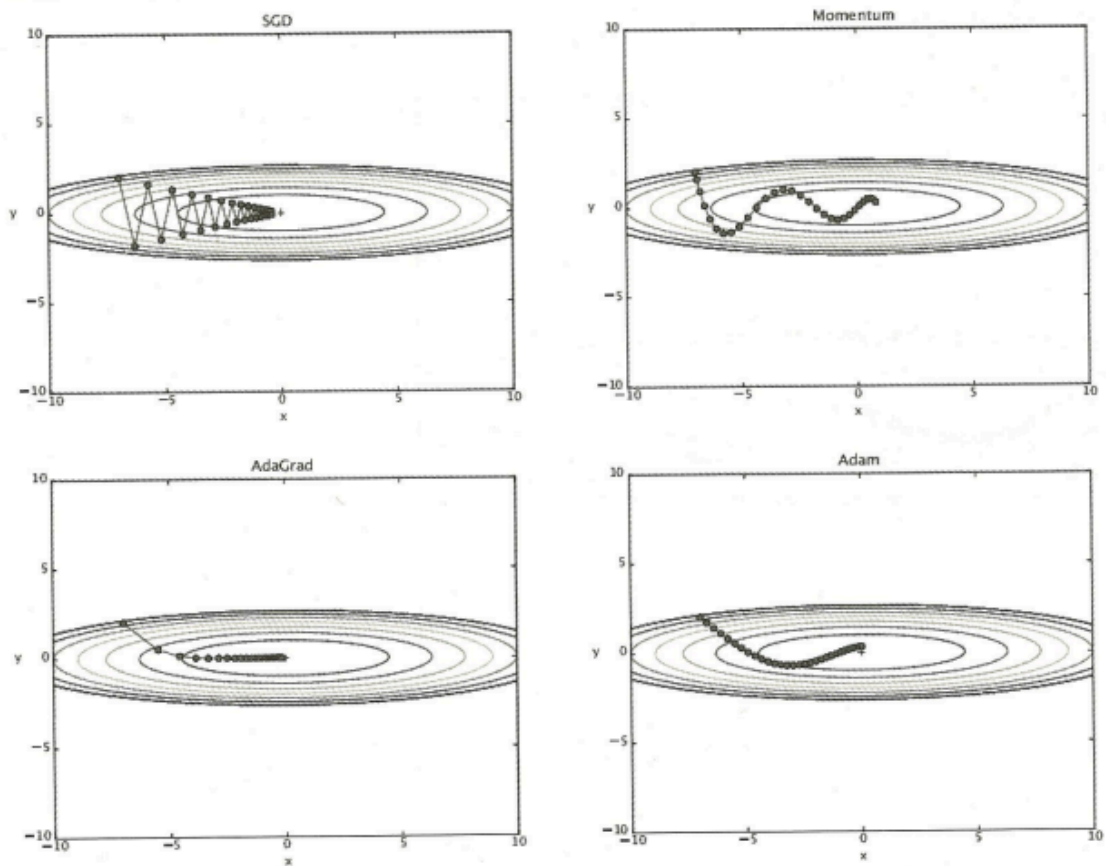
그림 6-7 Adam에 의한 최적화 갱신 경로



바닥을 구르듯 움직(모멘텀)이지만 흔들림이 적고 이는 학습의 갱신 강도를 적응적으로 조정 (AdaGrad)해서 얻는 장점이다.

정리)

그림 6-8 최적화 기법 비교 : SGD, 모멘텀, AdaGrad, Adam

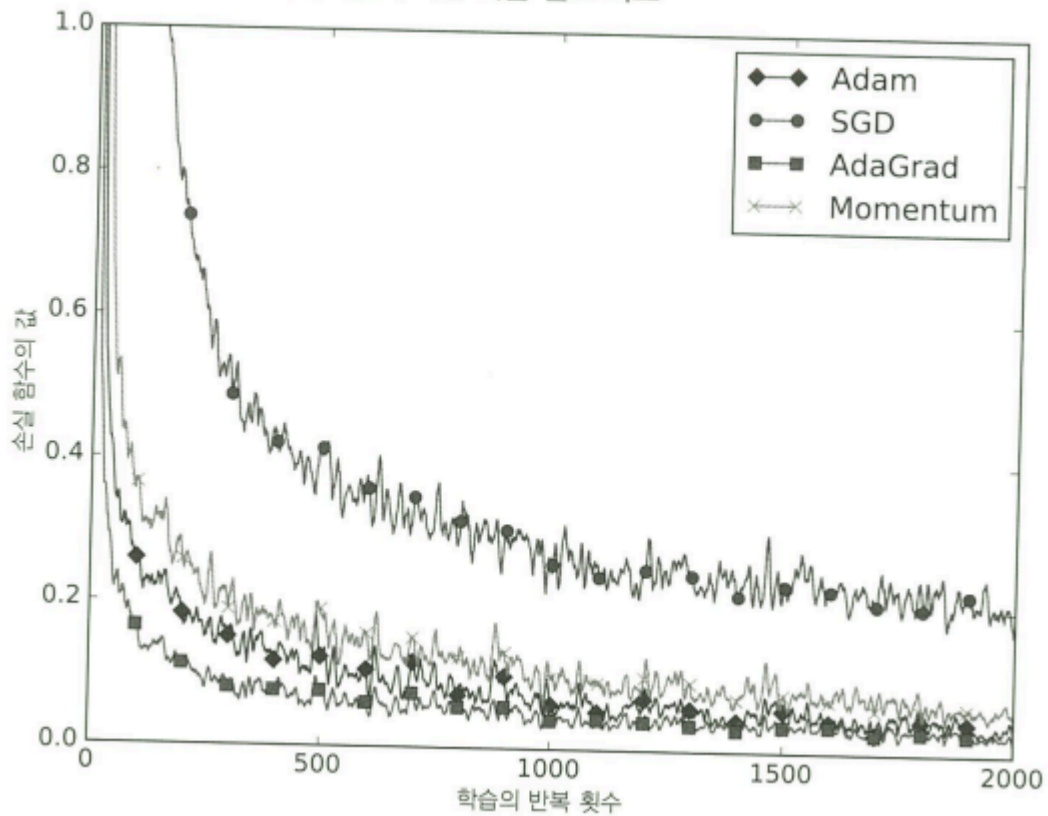


이중에서 가장 좋은 것은? 아직 완벽히 정해지지 않음. 각자의 장단점이 있기 때문

6.1.8. MNIST 데이터셋으로 본 갱신방법

⇒ MNIST에서는 어떤 방법이 좋을까

그림 6-9 MNIST 데이터셋에 대한 학습 진도 비교



100개의 뉴런 * 5층 * ReLU 활성화 함수

여기서 알 수 있는 것 :

SGD가 아직도 사용되고 있기는 하지만, 일반적으로 SGD보다 다른 3가지 기법들이 빠르게 학습한다

때로는 최종 정확도에서 차이가 나기도 한다

6.2. 가중치의 초기값

: 신경망 학습에서 가장 중요한 것

⇒ 권장 초기값은 무엇일까?

6.2.1. 초기값을 0으로

오버피팅을 억제하여 범용 성능을 높이는 테크닉 :

‘가중치 감소 기법’

⇒ 가중치 매개변수의 값이 작아지도록 학습하는 방법

⇒ 가중치 값을 작게하여 오버피팅이 일어나지 않게 해볼까?

→ 가중치 값을 작게 하면 오버피팅이 줄어드는 이유는 ****모델의 복잡도(표현력)****를 제한하기 때문

큰 가중치는 ****모델이 급격한 변화(복잡한 함수)****를 만드는데 입력이 조금만 바뀌어도 출력이 크게 바뀌게 되니, 일반화하기 어렵다.

가중치가 작으면 → 모델이 덜 민감함 → 덜 복잡함 → 오버피팅 감소

→ 지금까지 0.01배를 하였는데 0을 하면 어떨까?

⇒ 0으로 하면 학습이 안 된다.

⇒ 오차역전파법에서 모든 가중치의 값이 똑같이 갱신되기 때문

순전파로 가정했을 때 입력층의 가중치가 0이 되므로 두번째 층의 뉴런에 같은 값이 전달

같은 값이 전달된다는 말은 갱신을 거쳐도 업데이트가 안 된다는 말

⇒ 가중치를 여러개 가지는 의미 상실

⇒ 초기값은 무작위로 설정해야해

6.2.2. 은닉층의 활성화값 분포

가중치 초기값에 따른 은닉층 활성화값들의 변화

```

# coding: utf-8
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def ReLU(x):
    return np.maximum(0, x)

def tanh(x):
    return np.tanh(x)

input_data = np.random.randn(1000, 100) # 1000개의 데이터
node_num = 100 # 각 층의 노드(뉴런) 수
hidden_layer_size = 5 # 은닉층이 5개
activations = {} # 이곳에 활성화 결과를 저장

x = input_data

for i in range(hidden_layer_size):
    if i != 0:
        x = activations[i-1]

    # 초기값을 다양하게 바꿔가며 실험해보자!
    # w = np.random.randn(node_num, node_num) * 1
    # w = np.random.randn(node_num, node_num) * 0.01
    # w = np.random.randn(node_num, node_num) * np.sqrt(1.0 / node_num)
    w = np.random.randn(node_num, node_num) * np.sqrt(2.0 / node_num)

    a = np.dot(x, w)

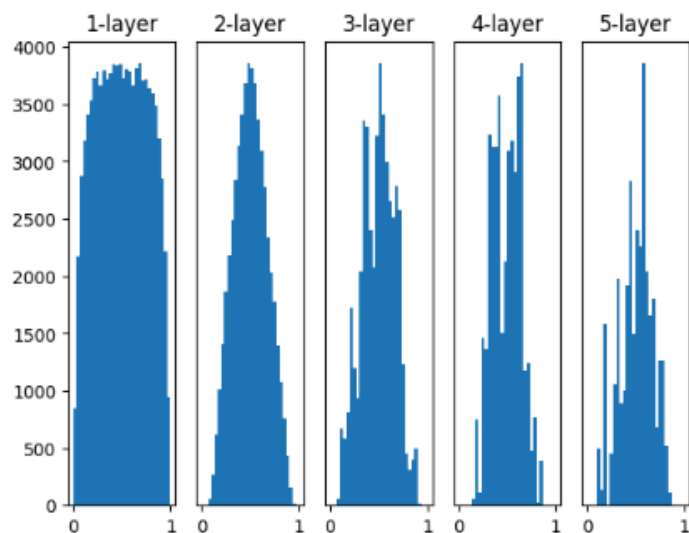
    # 활성화 함수도 바꿔가며 실험해보자!
    z = sigmoid(a)
    # z = ReLU(a)
    # z = tanh(a)

    activations[i] = z

# 히스토그램 그리기
for i, a in activations.items():
    plt.subplot(1, len(activations), i+1)
    plt.title(str(i+1) + "-layer")
    if i != 0:
        plt.yticks([], [])
    # plt.xlim(0.1, 1)
    # plt.ylim(0, 7000)
    plt.hist(a.flatten(), 30, range=(0, 1))

plt.show()

```



⇒ 초기값에 따라 신경망 학습의 성패가 갈린다는 내용

6.3. 배치 정규화

각 층이 활성화를 적당히 퍼트리도록 "강제"하기

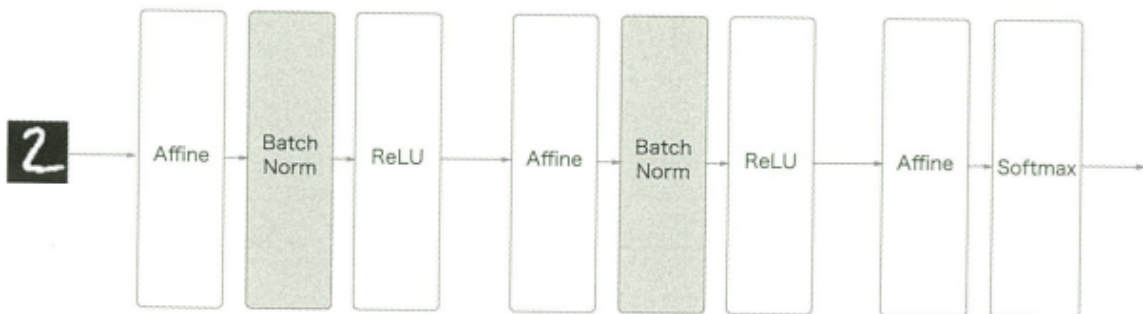
6.3.1. 배치 정규화 알고리즘

장점1. 학습 속도 개선

장점2. 초기값에 크게 의존하지 않음

장점3. 오버피팅 억제

그림 6-16 배치 정규화를 사용한 신경망의 예



활성화값이 적당히 분포되도록 조정하는 방법

1. 미니배치를 단위별로 정규화함

⇒ 데이터 분포가 평균:0, 분산:1으로 정규화

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

→ 입실론 값의 의미 : 0으로 나누는

사태 예방

⇒ 활성화 함수 앞 or 뒤에 삽입하면 데이터 분포가 덜 치우치도록 한다.

$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

⇒ 배치 정규화 계층마다 정규화된 데이터에 고유한 **확대**와 **이동 변환** 수행

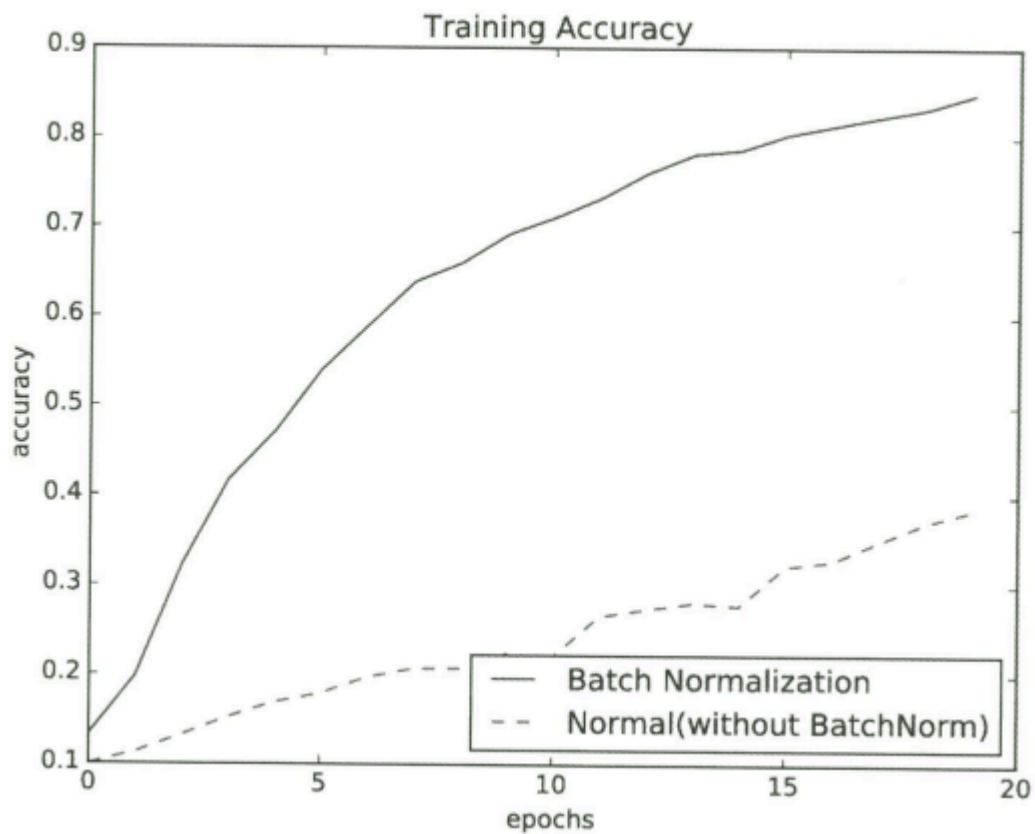
→ r이 확대, b가 이동을 담당

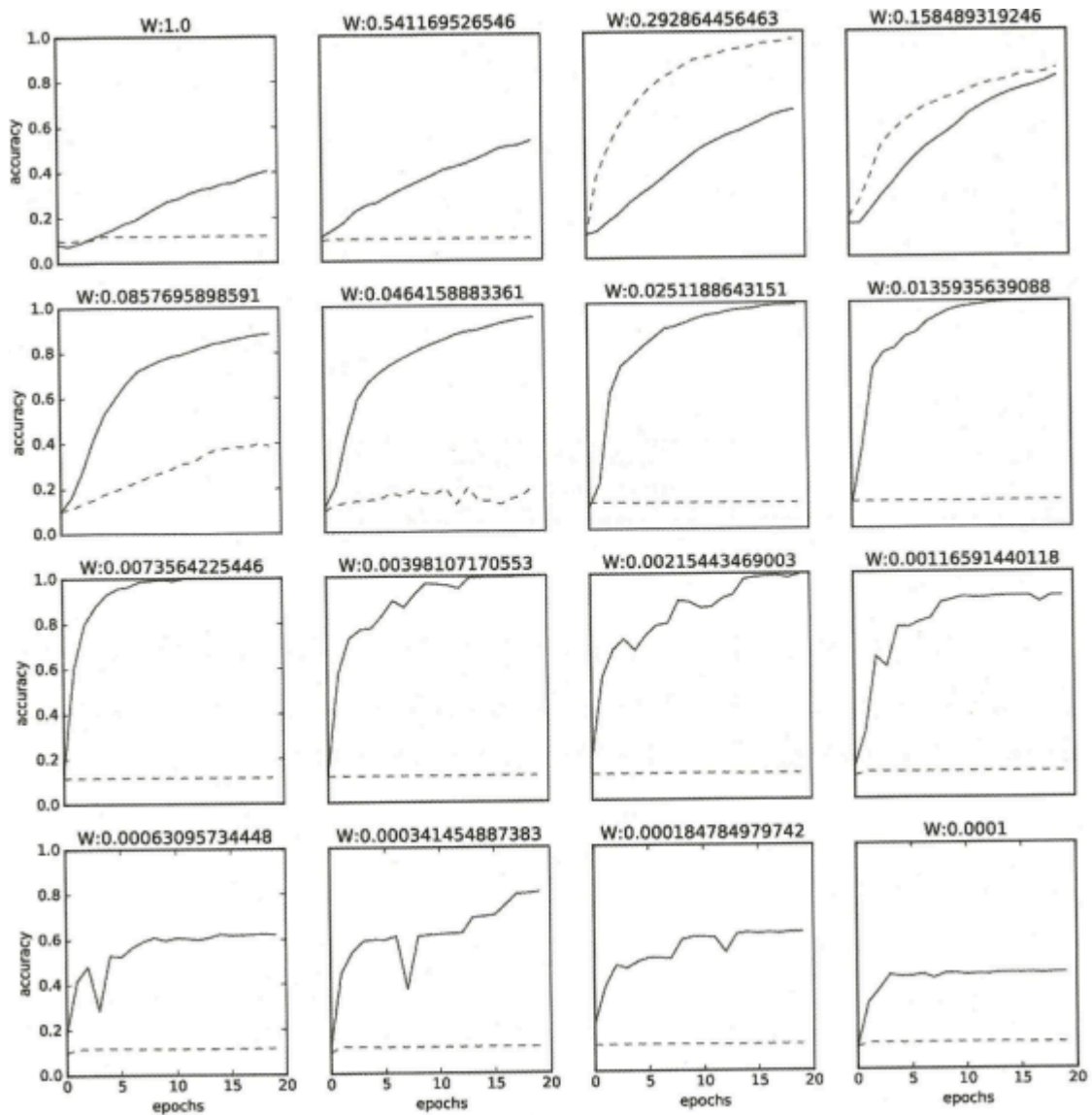
→ 초기값은 $r = 1, b = 0$

→ 학습 하며 적합한 값으로 조정

6.3.2. 배치 정규화 효과

그림 6-18 배치 정규화의 효과 : 배치 정규화가 학습 속도를 높인다.





6.4. 바른 학습을 위해

기계학습의 문제 : 오버피팅

→ 훈련 데이터에 지나치게 적응되어서 그 외의 데이터에 제대로 대응하지 못하는 상태

But, 기계학습이 지향하는 건 범용 성능

6.4.1. 오버피팅

발생하는 경우

1. 매개변수가 많고 표현력이 높음
2. 훈련 데이터가 적음

IF)

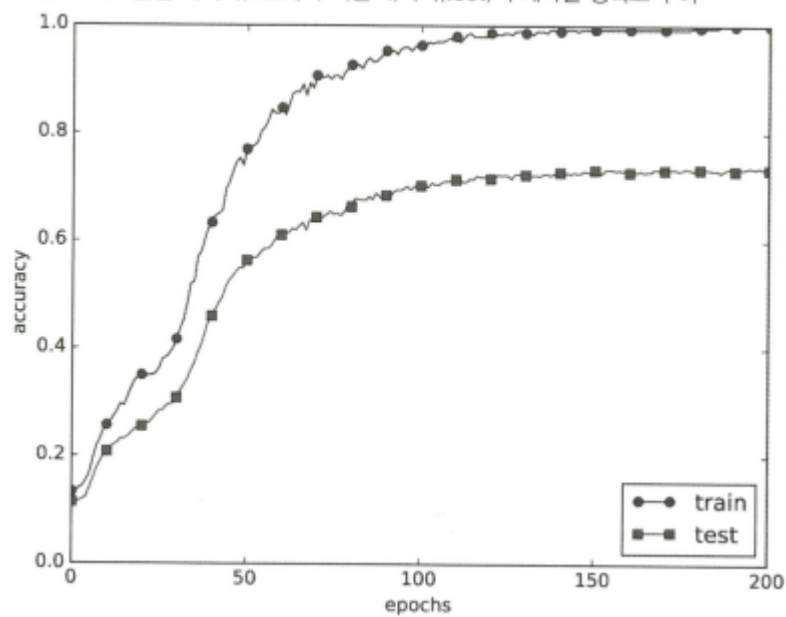
data의 갯수 : 60,000개의 MNIST 훈련 데이터 중 300개 ⇒ 데이터 갯수 ↓

네트워크 층 : 7층 ⇒ 복잡성 ↑

각 층의 뉴런 갯수 : 100개

활성화 함수 : ReLU

그림 6-20 훈련 데이터(train)와 시험 데이터(test)의 에폭별 정확도 추이

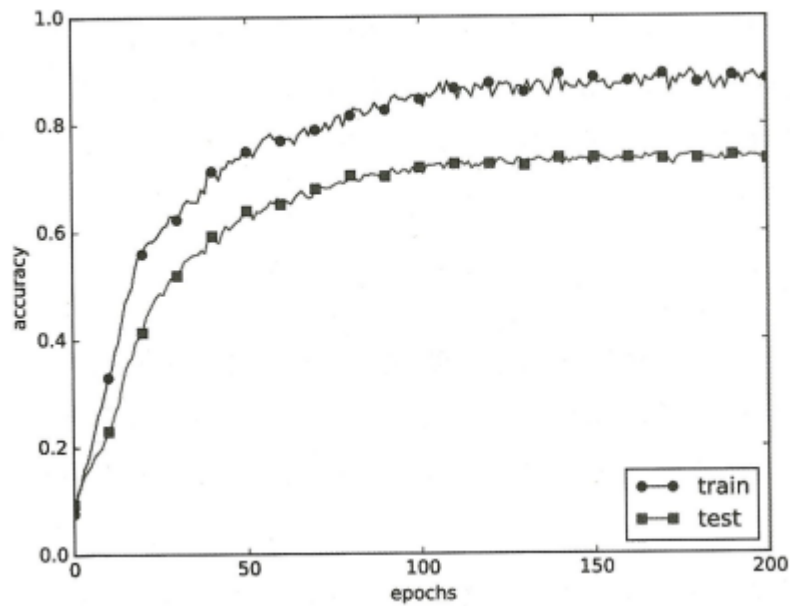


정확도의 차이가 매우 큼 ⇒ 오버피팅된 결과

6.4.2. 가중치 감소

⇒ for 오버피팅 억제

그림 6-21 가중치 감소를 이용한 훈련 데이터(train)와 시험 데이터(test)에 대한 정확도 추이



위와 비교했을 때 차이가 매우 준 것을 확인할 수 있음

AND, 훈련 데이터에 대한 정확도도 위와 달리 100%에 도달하지 못했다.

6.4.3. 드롭아웃

⇒ 뉴런을 임의로 삭제하며 학습

⇒ 훈련시 은닉층의 뉴런을 무작위로 골라서 삭제 → 삭제된 뉴런은 신호를 전달하지 X

⇒ data를 흘릴 때마다 삭제할 뉴런을 무작위로 선택, 시험(test)때 모든 뉴런에 신호를 전달

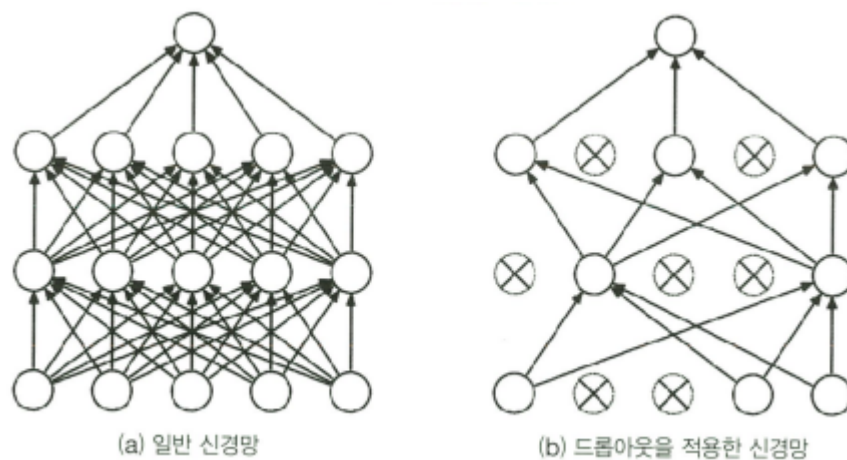
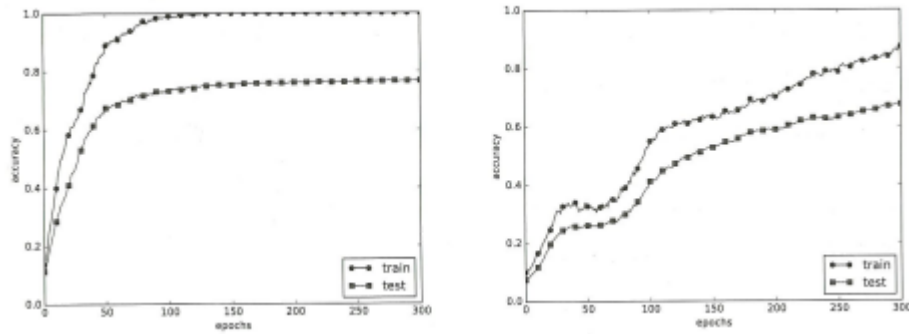


그림 6-23 왼쪽은 드롭아웃 없이, 오른쪽은 드롭아웃을 적용한 결과 (dropout_ratio = 0.15)



⇒ 표현력을 높이면서, 오버피팅을 억제

초기 epoch 혹은 train accuracy 기준으로 모델 성능을 평가하는 오류에서 나올 수 있는 착각이야. 드롭아웃의 목적은 오버피팅 방지, 일반화 성능을 높이기 위한 것. 다음의 그래프는 예시로서 적절치 않다고 봐야함

6.5. 적절한 하이퍼파라미터 값 찾기

하이퍼파라미터란?

- 1) 각 층의 뉴런 수
- 2) 배치 크기
- 3) 매개변수 갱신시 학습률
- 4) 매개변수 갱신시 가중치 감소

다음과 같은 하이퍼파라미터 값을 적절히 설정해야 모델의 성능이 좋다.

BUT, 결정 전까지는 많은 시행착오 발생

⇒ 하이퍼파라미터의 값을 최대한 효율적으로 탐색하는 방법은?

6.5.1. 검증 데이터

기존 교재 내용 : 훈련 데이터 + 시험 데이터

From now : 하이퍼파라미터 조정용 data를 "검증 데이터" 추가 (하이퍼파라미터 적절성 평가 데이터)

⇒ 훈련 데이터 + 검증 데이터 + 시험 데이터 → 오버피팅을 방지할 수 있음

가장 심플한 방법 :

훈련 data중 20%정도를 검증 data로 분리하는 것

6.5.2. 하이퍼파라미터 최적화

검증 데이터를 사용하여 하이퍼파라미터 최적화하기

최적화의 핵심 :

하이퍼파라미터의 "최적 값" 존재 범위를 줄여나가기

(= 좋은 성능을 내는 값이 존재할 것 같은 "범위"를 점점 좁혀가라)

과정 :

대략적인 범위 설정 → 범위에서 무작위로 하이퍼파라미터값 고르기 → 정확도 평가
⇒ 정확도를 살피며 계속 반복 → 하이퍼파라미터의 "최적값" 범위 좁히기
⇒ 에폭을 작게해서 1회 평가에 걸리는 시간을 단축하는 것이 효과적

- 0단계
하이퍼파라미터 값의 범위를 설정합니다.
- 1단계
설정된 범위에서 하이퍼파라미터의 값을 무작위로 추출합니다.
- 2단계
1단계에서 샘플링한 하이퍼파라미터 값을 사용하여 학습하고, 검증 데이터로 정확도를 평가합니다(단, 에폭은 작게 설정합니다).
- 3단계
1단계와 2단계를 특정 횟수(100회 등) 반복하며, 그 정확도의 결과를 보고 하이퍼파라미터의 범위를 좁힙니다.

이렇게 하이퍼파라미터의 범위를 좁히고, 좁혀지면 압축한 범위에서 값을 골라낸다.

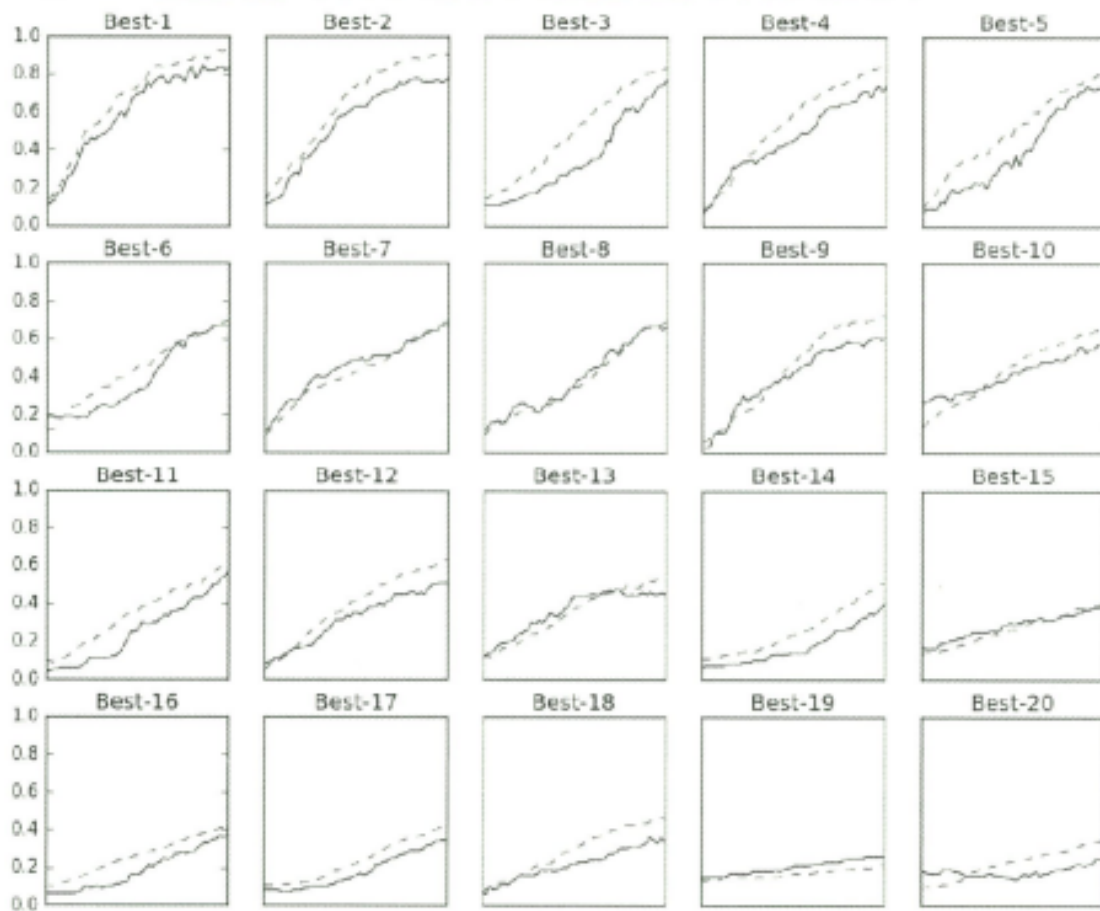
⇒ 좀 더 수학적인 이유를 원한다면?

⇒ 베이지스 최적화

6.5.3. 하이퍼파라미터 최적화 구현하기

MNIST 데이터셋을 활용하여 학습률과 가중치 감소 세기 조절 계수 탐색해보기

그림 6-24 실선은 검증 데이터에 대한 정확도, 점선은 훈련 데이터에 대한 정확도



```
Best-1 (val acc:0.83) | lr:0.0092, weight decay:3.86e-07
Best-2 (val acc:0.78) | lr:0.00956, weight decay:6.04e-07
Best-3 (val acc:0.77) | lr:0.00571, weight decay:1.27e-06
Best-4 (val acc:0.74) | lr:0.00626, weight decay:1.43e-05
Best-5 (val acc:0.73) | lr:0.0052, weight decay:8.97e-06
```

Best 1~5 까지는 성능이 괜찮다고 할 수 있음

6.6. 정리

1. 매개변수 갱신 방법
2. 가중치 초기값 설정 방법
3. 배치 정규화

4. 드롭아웃

이번 장에서 배운 내용

- 매개변수 갱신 방법에는 확률적 경사 하강법(SGD) 외에도 모멘텀, AdaGrad, Adam 등이 있다.
- 가중치 초기값을 정하는 방법은 올바른 학습을 하는 데 매우 중요하다.
- 가중치의 초기값으로는 'Xavier 초기값'과 'He 초기값'이 효과적이다.
- 배치 정규화를 이용하면 학습을 빠르게 진행할 수 있으며, 초기값에 영향을 덜 받게 된다.
- 오버피팅을 억제하는 정규화 기술로는 가중치 감소와 드롭아웃이 있다.
- 하이퍼파라미터 값 탐색은 최적 값이 존재할 법한 범위를 점차 좁히면서 하는 것이 효과적이다.